

AHA MOMENT #1: THE DUAL REPRESENTATION (TREE ↔ ARRAY)

A Heap is conceptually a **Complete Binary Tree**, but physically stored as a flat 1D array with zero pointers! Node addresses are computed via $O(1)$ arithmetic: $\text{parent}(i) = (i - 1) / 2$, $\text{left}(i) = 2 * i + 1$, $\text{right}(i) = 2 * i + 2$.

REUSABLE HEAP (PRIORITYQUEUE) UTILITIES & CONVERSIONS

1. Declarations & Comparators:

```
// Min-Heap (Default)
PriorityQueue<Integer> minHeap = new PriorityQueue<>();

// Max-Heap (Collections.reverseOrder)
PriorityQueue<Integer> maxHeap = new PriorityQueue<>
(Collections.reverseOrder());

// Pair Comparator: (a, b) → Integer.compare(a[1], b[1])
PriorityQueue<int[]> pairPq = new PriorityQueue<>((a, b) →
Integer.compare(a[1], b[1]));
```

2. Top-K Window Insertion Template:

```
// Maintain Top K Largest items in O(N log K) time
PriorityQueue<Integer> pq = new PriorityQueue<>();
for (int num : nums) {
    pq.offer(num);
    if (pq.size() > k) pq.poll(); // Evict smallest root!
}
// pq contains exactly top K largest elements!
```

1 WHAT IS A HEAP?

- **Complete Binary Tree:** All levels filled left-to-right.
- **Weak Ordering:** Parent vs Child property only (Root is absolute min or max).
- **Priority Access:** Instant $O(1)$ access to root, $O(\log N)$ insert & remove.
- **Heap vs BST:** BST is fully sorted ($O(\log N)$ search). Heap is weakly ordered, eliminating tree balancing overhead!

2 MIN-HEAP VS MAX-HEAP PROPERTY

MIN HEAP

Parent $\leq$ Children
Root = Global Minimum

MAX HEAP

Parent $\geq$ Children
Root = Global Maximum

3 0-INDEXED ARRAY MAPPING & TIME COMPLEXITIES

Operation	Formula / Method	Time Complexity	Why?
Parent Index	$(i - 1) / 2$	$O(1)$	Bitwise / integer division
Left Child Index	$2 * i + 1$	$O(1)$	Left child offset
Right Child Index	$2 * i + 2$	$O(1)$	Right child offset
Peek Root	<code>peek()</code>	$O(1)$	Read element at index 0
Insert Element	<code>offer(x)</code>	$O(\log N)$	Append at end & Heapify Up
Poll Root	<code>poll()</code>	$O(\log N)$	Swap root with last & Heapify Down
Build Heap	<code>new PriorityQueue(list)</code>	$O(N)$	Floyd's bottom-up heapify algorithm

★ **GOLDEN RULE:** Heaps are all about PRIORITY, not sorting! Ask: "What element should come out first?"

AHA MOMENT #2: HEAPIFY UP VS SINK DOWN & FLOYD'S O(N) BUILD

- **Heapify Up (Bubble Up):** Swaps newly inserted leaf with parent until `parent <= child` ($O(\log N)$).
- **Heapify Down (Sink Down):** Swaps root with smaller child after polling ($O(\log N)$).
- **Floyd's $O(N)$ Build Heap:** Calling heapify down starting from the last non-leaf node `(n/2 - 1)` down to 0 builds a heap in $O(N)$ linear time!

1
JAVA PRIORITYQUEUE: 3 COMPARATOR PATTERNS

```
// 1. Min Heap (Default)
PriorityQueue<Integer> minHeap = new PriorityQueue<>();

// 2. Max Heap (Collections.reverseOrder)
PriorityQueue<Integer> maxHeap =
    new PriorityQueue<>(Collections.reverseOrder());

// 3. Max Heap (Safe Integer.compare)
PriorityQueue<Integer> maxHeap =
    new PriorityQueue<>((a, b) -> Integer.compare(b, a));
```

2

COMMON INTERVIEW TRAPS & MISTAKES

- ✗ **Subtraction Overflow:** `(a,b)→a-b` overflows when subtracting negative numbers! Always use `Integer.compare(a, b)`.
- ✗ **Linear Removal Trap:** `pq.remove(obj)` takes $O(N)$ time because it scans the underlying array!
- ✗ **Assuming Sorted Output:** Iterating over a `PriorityQueue` via `for-each` loop DOES NOT give sorted order! You must call `poll()` sequentially.

3
PRIORITYQUEUE SAFE API CHEAT SHEET

Method	Description	Time Complexity	Empty Heap Behavior
<code>offer(x)</code>	Inserts element `x` into heap	$O(\log N)$	Returns `false` if restricted capacity fails
<code>poll()</code>	Removes & returns root (min/max)	$O(\log N)$	Returns `null` if empty
<code>peek()</code>	Returns root without removing	$O(1)$	Returns `null` if empty
<code>size()</code>	Returns total elements in heap	$O(1)$	Returns `0`
<code>isEmpty()</code>	Checks if heap has zero elements	$O(1)$	Returns `true`

💡 Every Heap problem in FAANG interviews reduces to one of these 4 core design patterns!

PATTERN 1: REPEATED MAX/MIN

- Repeatedly extract extreme element.
- Process element & re-insert.
- Max-Heap for largest (e.g. LC 1046 Last Stone Weight).

```
PriorityQueue<Integer> pq = new PriorityQueue<>
(Collections.reverseOrder());
for (int s : stones) pq.offer(s);
while (pq.size() > 1) {
    int a = pq.poll(), b = pq.poll();
    if (a != b) pq.offer(a - b);
}
```

Time: $O(N \log N)$
Space: $O(N)$
Target: LC 1046

PATTERN 2: TOP-K ELEMENTS

- Need K largest/smallest elements.
- Top K Largest → **Min-Heap of size K**.
- Top K Smallest → **Max-Heap of size K**.

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
for (int num : nums) {
    pq.offer(num);
    if (pq.size() > k) pq.poll();
}
// pq contains K largest items
```

Time: $O(N \log K)$
Space: $O(K)$
Target: LC 215, 347, 973

PATTERN 3: MERGE K SORTED STREAMS

- K sorted arrays/lists.
- Offer 1st node of each stream into Min-Heap.
- Poll min, offer 'next' node.

```
PriorityQueue<ListNode> pq = new PriorityQueue<>((a,b)-
>Integer.compare(a.val,b.val));
for (ListNode h : lists) if (h != null) pq.offer(h);
while (!pq.isEmpty()) {
    ListNode curr = pq.poll();
    tail.next = curr; tail = tail.next;
    if (curr.next != null) pq.offer(curr.next);
}
```

Time: $O(N \log K)$
Space: $O(K)$
Target: LC 23, 373

PATTERN 4: TWO HEAPS (MEDIAN)

- Continuous data stream.
- 'maxHeap' (lower half) + 'minHeap' (higher half).
- Rebalance to keep size diff ≤ 1 .

```
if (maxHeap.isEmpty() || num <= maxHeap.peek())
    maxHeap.offer(num);
else minHeap.offer(num);
if (maxHeap.size() > minHeap.size() + 1)
    minHeap.offer(maxHeap.poll());
else if (minHeap.size() < maxHeap.size())
    maxHeap.offer(minHeap.poll());
```

Insert: $O(\log N)$
Median: $O(1)$
Target: LC 295

🔍 QUESTION SIGNAL → HEAP PATTERN MAP

Question Keyword / Phrase	Heap Pattern to Choose
"Find Top K / Kth Largest"	Min-Heap of size K
"K Closest Points to Origin"	Max-Heap of size K
"Merge K sorted lists"	Min-Heap of size K streams
"Continuous data stream median"	Two Heaps (Max + Min)
"Task Scheduler / CPU Cooldown"	Max-Heap Frequency + Cooldown Queue
"Maximize Capital / IPO"	Min-Heap (Capital) + Max-Heap (Profit)

🌿 HEAP DECISION TREE (MERMAID)



1. LEETCODE 1046 — LAST STONE WEIGHT
 **Easy** ★★★★★

Max-Heap Simulation
Amazon

Problem: Smash two heaviest stones together until at most 1 stone remains.

```
public int lastStoneWeight(int[] stones) {
    PriorityQueue<Integer> maxHeap = new PriorityQueue<>
        (Collections.reverseOrder());
    for (int s : stones) maxHeap.offer(s);
    while (maxHeap.size() > 1) {
        int y = maxHeap.poll(), x = maxHeap.poll();
        if (y != x) maxHeap.offer(y - x);
    }
    return maxHeap.isEmpty() ? 0 : maxHeap.peek();
}
```

2. LEETCODE 703 — KTH LARGEST ELEMENT IN A STREAM
 **Easy** ★★★★★

Min-Heap Size K
Amazon

Problem: Design class to find K^{th} largest element in a stream.

```
class KthLargest {
    private PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    private int k;
    public KthLargest(int k, int[] nums) {
        this.k = k;
        for (int num : nums) add(num);
    }
    public int add(int val) {
        minHeap.offer(val);
        if (minHeap.size() > k) minHeap.poll();
        return minHeap.peek(); // Root is Kth largest!
    }
}
```

3. LEETCODE 973 — K CLOSEST POINTS TO ORIGIN
 **Medium** ★★★★★

Max-Heap Size K
Amazon
Meta
Problem: Find K points closest to $(0,0)$ using Euclidean distance.

```
public int[][] kClosest(int[][] points, int k) {
    PriorityQueue<int[]> maxHeap = new PriorityQueue<
        (a, b) → Integer.compare((b[0]*b[0] + b[1]*b[1]), (a[0]*a[0] +
        a[1]*a[1]))
    >();
    for (int[] p : points) {
        maxHeap.offer(p);
        if (maxHeap.size() > k) maxHeap.poll();
    }
    int[][] res = new int[k][2];
    for (int i = 0; i < k; i++) res[i] = maxHeap.poll();
    return res;
}
```

4. LEETCODE 347 — TOP K FREQUENT ELEMENTS
 **Medium** ★★★★★

HashMap + Min-Heap
Amazon
Google
Problem: Return K most frequent elements in array.

```
public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> count = new HashMap<>();
    for (int num : nums) count.put(num, count.getOrDefault(num, 0) + 1);
    PriorityQueue<Map.Entry<Integer, Integer>> minHeap =
        new PriorityQueue<((a, b) → Integer.compare(a.getValue(),
        b.getValue()))>();
    for (Map.Entry<Integer, Integer> entry : count.entrySet()) {
        minHeap.offer(entry);
        if (minHeap.size() > k) minHeap.poll();
    }
    int[] res = new int[k];
    for (int i = 0; i < k; i++) res[i] = minHeap.poll().getKey();
    return res;
}
```

5. LEETCODE 621 — TASK SCHEDULER

Medium ★★★★★

Max-Heap + Cooldown Queue

Meta

Problem: Minimum CPU cycles to finish tasks with N cooldown constraint.

```
public int leastInterval(char[] tasks, int n) {
    int[] freq = new int[26];
    for (char c : tasks) freq[c - 'A']++;
    PriorityQueue<Integer> maxHeap = new PriorityQueue<>
(Collections.reverseOrder());
    for (int f : freq) if (f > 0) maxHeap.offer(f);
    Queue<int[]> cooldown = new ArrayDeque<>(); // [freq, availTime]
    int time = 0;
    while (!maxHeap.isEmpty() || !cooldown.isEmpty()) {
        time++;
        if (!maxHeap.isEmpty()) {
            int cnt = maxHeap.poll() - 1;
            if (cnt > 0) cooldown.offer(new int[]{cnt, time + n});
        }
        if (!cooldown.isEmpty() && cooldown.peek()[1] == time) {
            maxHeap.offer(cooldown.poll()[0]);
        }
    }
    return time;
}
```

6. LEETCODE 295 — FIND MEDIAN FROM DATA STREAM

Hard ★★★★★

Two Heaps (Max + Min)

Google

Amazon

Problem: Maintain running median of a stream in $O(\log N)$ insert & $O(1)$ find.

```
class MedianFinder {
    private PriorityQueue<Integer> maxHeap = new PriorityQueue<>
(Collections.reverseOrder());
    private PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    public void addNum(int num) {
        if (maxHeap.isEmpty() || num <= maxHeap.peek())
            maxHeap.offer(num);
        else minHeap.offer(num);
        if (maxHeap.size() > minHeap.size() + 1)
            minHeap.offer(maxHeap.poll());
        else if (minHeap.size() < maxHeap.size())
            maxHeap.offer(minHeap.poll());
    }
    public double findMedian() {
        if (maxHeap.size() == minHeap.size()) return (maxHeap.peek() +
minHeap.peek()) / 2.0;
        return maxHeap.peek();
    }
}
```

AHA MOMENT: CANDIDATE TRACKING ACROSS K SORTED STREAMS

When merging K sorted inputs (lists, arrays, or matrix rows), you only ever need to hold ≤ 1 active candidate from each of the K sources inside a Min-Heap of size K !

7. LEETCODE 23 — MERGE K SORTED LISTS

Hard ★★★★★

Min-Heap Size K

Amazon

Meta

Problem: Merge K sorted linked lists into one single sorted list.

Key Strategy: Insert the head of each non-null list into a Min-Heap of size K .

Continuously poll the minimum node, append it to the result list, and if that node has a `next` pointer, offer `next` into the heap!

```
public ListNode mergeKLists(ListNode[] lists) {
    if (lists == null || lists.length == 0) return null;
    PriorityQueue<ListNode> pq = new PriorityQueue<>((a,b) ->
        Integer.compare(a.val, b.val));
    for (ListNode node : lists) if (node != null) pq.offer(node);
    ListNode dummy = new ListNode(0), tail = dummy;
    while (!pq.isEmpty()) {
        ListNode curr = pq.poll();
        tail.next = curr; tail = tail.next;
        if (curr.next != null) pq.offer(curr.next);
    }
    return dummy.next;
}
```

7B. LEETCODE 373 — FIND K PAIRS WITH SMALLEST SUMS

Medium ★★★★★

Min-Heap Index Tracking

Google

Problem: Given two sorted arrays `u` and `v`, find K pairs $(u[i], v[j])$ with smallest sums.

```
public List<List<Integer>> kSmallestPairs(int[] nums1, int[] nums2, int k) {
    List<List<Integer>> res = new ArrayList<>();
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[0] + a[1], b[0] + b[1]));
    for (int i = 0; i < Math.min(nums1.length, k); i++) {
        pq.offer(new int[]{nums1[i], nums2[0], 0}); // [u, v, idxInNums2]
    }
    while (k-- > 0 && !pq.isEmpty()) {
        int[] curr = pq.poll();
        res.add(List.of(curr[0], curr[1]));
        int idx2 = curr[2];
        if (idx2 + 1 < nums2.length) {
            pq.offer(new int[]{curr[0], nums2[idx2 + 1], idx2 + 1});
        }
    }
    return res;
}
```


 AHA MOMENT: DUAL HEAP / SLIDING WINDOW HEAP DYNAMICS

Advanced Heap problems combine a Min-Heap (to track the current minimum element across streams) with dynamic tracking of the current maximum element to evaluate ranges or maximize capital!

8. LEETCODE 502 — IPO (MAXIMIZE CAPITAL)

 **Hard ★★★★★**

Dual Heap Greedy

 

Problem: Pick at most K projects to maximize initial capital W .

```
public int findMaximizedCapital(int k, int w, int[] profits, int[] capital) {
    int n = profits.length;
    int[][] projects = new int[n][2];
    for (int i = 0; i < n; i++) projects[i] = new int[]{capital[i], profits[i]};
    Arrays.sort(projects, (a,b) → Integer.compare(a[0], b[0]));
    PriorityQueue<Integer> maxProfit = new PriorityQueue<>
        (Collections.reverseOrder());
    int ptr = 0;
    for (int i = 0; i < k; i++) {
        while (ptr < n && projects[ptr][0] ≤ w) {
            maxProfit.offer(projects[ptr++][1]);
        }
        if (maxProfit.isEmpty()) break;
        w += maxProfit.poll();
    }
    return w;
}
```

9. LEETCODE 632 — SMALLEST RANGE COVERING ELEMENTS FROM K LISTS

 **Hard ★★★★★**

Min-Heap + Max Tracker

 

 

Problem: Find smallest range $[a, b]$ that includes at least one number from each of the K sorted lists.

```
public int[] smallestRange(List<List<Integer>> nums) {
    PriorityQueue<int> minHeap = new PriorityQueue<>
        (a, b) → Integer.compare(a[0], b[0]);
    int maxVal = Integer.MIN_VALUE;
    for (int i = 0; i < nums.size(); i++) {
        int val = nums.get(i).get(0);
        minHeap.offer(new int[]{val, i, 0}); // [val, listIdx, elemIdx]
        maxVal = Math.max(maxVal, val);
    }
    int rangeStart = 0, rangeEnd = Integer.MAX_VALUE;
    while (minHeap.size() == nums.size()) {
        int[] curr = minHeap.poll();
        int minVal = curr[0], listIdx = curr[1], elemIdx = curr[2];
        if (maxVal - minVal < rangeEnd - rangeStart) {
            rangeStart = minVal; rangeEnd = maxVal;
        }
        if (elemIdx + 1 < nums.get(listIdx).size()) {
            int nextVal = nums.get(listIdx).get(elemIdx + 1);
            minHeap.offer(new int[]{nextVal, listIdx, elemIdx + 1});
            maxVal = Math.max(maxVal, nextVal);
        }
    }
    return new int[]{rangeStart, rangeEnd};
}
```

📋 REVISION CHECKLIST

- ✔ **Dual Representation:** Conceptual Complete Binary Tree ↔ Flat 1D Array ($\text{parent}(i) = (i-1)/2$).
- ✔ **Top-K Min-Heap Rule:** K Largest → Min-Heap of size K ($O(N \log K)$ time, $O(K)$ space).
- ✔ **Merge K Streams:** Min-Heap of size K holding current head node of each stream.
- ✔ **Two-Heap Median:** `maxHeap` (lower half) + `minHeap` (higher half) kept balanced within 1 element.
- ✔ **Comparator Safety:** Always use `Integer.compare(a, b)` instead of `a - b` to prevent overflow!

🚀 Next Step: Move to **Topic 10: Trees & BST Masterclass**.